1. Виконавець: Грушовий Максим Олександрович, група К-25. Програмний проєкт №4, варіант №11.

2. Умова (варіант №11, m=3): структура з 3 цілих полів (звичайний масив розміру 3) з операціями get/set для кожного поля та operator string для всієї структури; без atomic. Частоти операцій від усіх: поле 0 — read 10%, write 5%; поле 1 — read 10%, write 5%; поле 2 — read 10%, write 20%; string 40%. Три файли: а) за умовою, б) рівні частоти, в) вручну підібрані (80% write поля 0, 9% write поля 1, 7% write поля 2, string 0,5%, решта read). Виміри для 1, 2, 3 потоків; загальна кількість операцій (3 000 000) однакова, кожен потік виконує власну неперервну частину послідовності.

3. Схема захисту та обґрунтування. Використано **3 м'ютекси типу std::shared_mutex — по одному на кожне поле** (кожен захищає лише своє поле). get(i) бере shared_lock лише на м'ютекс i, set(i) — unique_lock лише на м'ютекс i, тому операції над різними полями не блокують одна одну. operator string бере shared_lock на всі три м'ютекси *завжди в порядку 0→1→2*, щоб отримати узгоджений знімок; інші операції тримають щонайбільше один м'ютекс, тож цикл очікування неможливий і взаємоблокувань немає. Значення копіюються під замками, а рядок формується після їх відпускання — критична секція мінімальна. Чому саме так: операції читання (read + string, що читає всі поля) становлять 50% для полів 0 і 1 та 50% для поля 2 проти записів 5%, 5% і 20% відповідно, тобто всі поля «читацькі», тому shared-режим дозволяє читачам (особливо частим string) працювати одночасно. Один спільний м'ютекс серіалізував би навіть доступ до різних полів, а м'ютекс на кожне поле усуває цей конфлікт. Слоти вирівняно по 64 байти (alignas), щоб м'ютекси різних полів не ділили рядок кешу (false sharing).

4. Середній час виконання (мс, середнє з 5 запусків, 3 000 000 операцій, Release x64, /O2):

Файл	**1 потік**	**2 потоки**	**3 потоки**
а) частоти за умовою	153,88	235,67	359,68
б) усі частоти рівні	91,11	131,69	184,22
в) дуже відмінні від умови	51,89	70,67	95,92

5. Висновки. Виміри виконано на: Intel Core i7-13620H, 10 ядер / 16 логічних процесорів, Windows, Visual Studio, Release x64. Очікування: оскільки загальна кількість операцій фіксована, а потоки працюють з різними полями й переважно з read-блокуваннями, час мав би спадати зі зростанням кількості потоків. Фактично: для а) час зростає (у 2,34 раза при 3 потоках), для б) у 2,02 раза, для в) у 1,85 раза. Тобто очікуване прискорення не з'явилося: кожна операція триває лише десятки наносекунд, тому корисної роботи, яку можна розпаралелити, майже немає, а всі потоки постійно змінюють стан тих самих трьох м'ютексів (навіть shared_lock змінює лічильник читачів), через що рядки кешу переходять між ядрами. Перевірена альтернатива зі звичайним std::mutex дала подібні результати: на 1–2 потоках shared_mutex швидший на кілька відсотків, на 3 потоках для файлу а) трохи повільніший; суттєвої різниці немає. Ядер достатньо (10 фізичних), тому причина не в апаратних обмеженнях, а в самій структурі навантаження.

6. Що зроблено самостійно. Налаштовано проєкти Visual Studio (стандарт C++20, Release x64, /O2, /utf-8), зібрано й запущено генератор та програму вимірювання; проведено вимірювання для трьох тестових файлів на 1, 2 і 3 потоках; додатково порівняно схему на shared_mutex зі схемою на звичайному std::mutex; проаналізовано результати та оформлено звіт.